Digital Bib

Plateforme de ressources académiques en ligne

Med Dhia Chtioui Iheb Zarrouk Mohamed Touzi
Aziz Chargui Yassin Oumaya

Établissement : ISIMM — Université de Monastir
Enseignante : Senda Laatawi
Année universitaire : 2025–2026

22 avril 2026

Table des matières

1 Introduction et présentation de l'idée **3**
1.1 Contexte et objectifs . . . 3
1.2 L'idée originale . . . 3
1.3 Défense de l'idée . . . 3

2 Conception **5**
2.1 Diagramme des cas d'utilisation principal . . . 5
2.2 Détail des cas d'utilisation . . . 7
2.2.1 UC1 : Rechercher une ressource . . . 7
2.2.2 UC2 : Télécharger une ressource . . . 7
2.2.3 UC3 : S'authentifier . . . 7
2.3 Diagrammes de séquence . . . 9
2.3.1 Recherche filtrée de ressources . . . 9
2.3.2 Authentification . . . 10
2.4 Diagramme de classes . . . 11
2.5 Architecture technique . . . 11
2.6 Maquettes / Wireframes . . . 12

3 Développement **13**
3.1 Technologies choisies et justifications . . . 13
3.2 Organisation du code . . . 13
3.3 Implémentation des fonctionnalités clés . . . 13
3.3.1 Recherche et filtrage des ressources . . . 13
3.3.2 Authentification . . . 14
3.3.3 Gestion des fichiers téléchargés . . . 15
3.4 Gestion des données . . . 15
3.4.1 Schéma de la base de données . . . 15

4 Tests et validation **17**
4.1 Environnement et outils de test . . . 17
4.2 Scénarios de test . . . 17

5 Conclusion et perspectives **18**
5.1 Bilan du projet . . . 18
5.2 Difficultés et apprentissages . . . 18
5.3 Évolutions possibles . . . 18

6 Annexes **19**
6.1 Lien vers le dépôt Git 19
6.2 Tableau récapitulatif des endpoints API 19
6.3 Fichiers de configuration 19

Chapitre 1

Introduction et présentation de l'idée

1.1 Contexte et objectifs

Dans le contexte de la transformation numérique de l'enseignement supérieur, les étudiants universitaires peinent à accéder de manière centralisée et organisée aux ressources pédagogiques (cours, travaux dirigés, séries d'exercices, sujets d'examens).

Le projet **Digital Bib** s'inscrit dans ce cadre en proposant une bibliothèque numérique académique destinée aux étudiants de l'ISIMM. Il vise à centraliser l'ensemble des ressources par section et par niveau d'études, tout en offrant des fonctionnalités de recherche avancée, de téléchargement et de suivi personnalisé.

1.2 L'idée originale

Digital Bib est une application web qui permet aux étudiants de :

- Consulter et filtrer des ressources pédagogiques (cours, TD, examens, exercices) par section (Sciences, Lettres, Droit, Économie) et par niveau (Licence 1 à Doctorat).
- Effectuer des recherches textuelles dans le catalogue de ressources.
- Télécharger et gérer une liste personnelle de fichiers installés.
- Suivre leur historique de recherche (limité à 20 entrées par utilisateur).
- S'authentifier via un système de login sécurisé avec gestion des rôles (étudiant / administrateur).

Digital Bib centralise les ressources académiques dispersées en une seule plateforme intuitive, accessible depuis n'importe quel navigateur, sans installation préalable.

1.3 Défense de l'idée

Le choix de ce projet est motivé par plusieurs constats :

- Les ressources pédagogiques sont souvent éparpillées entre groupes WhatsApp, clés USB et e-mails — Digital Bib élimine ces frictions.
- Le système de filtrage multi-critères (section, niveau, catégorie, mot-clé) réduit considérablement le temps de recherche.
- La gestion des fichiers téléchargés et l'historique de recherche apportent une expérience utilisateur personnalisée.

— Le modèle d'abonnement pour les utilisateurs étrangers ouvre des perspectives de viabilité économique.

Chapitre 2

Conception

2.1 Diagramme des cas d'utilisation principal

Le système **Digital Bib** implique deux acteurs principaux :

— **Étudiant** : rechercher et filtrer des ressources, les télécharger, gérer son profil et son historique.

— **Administrateur** : gérer les utilisateurs, les ressources et les codes académiques.

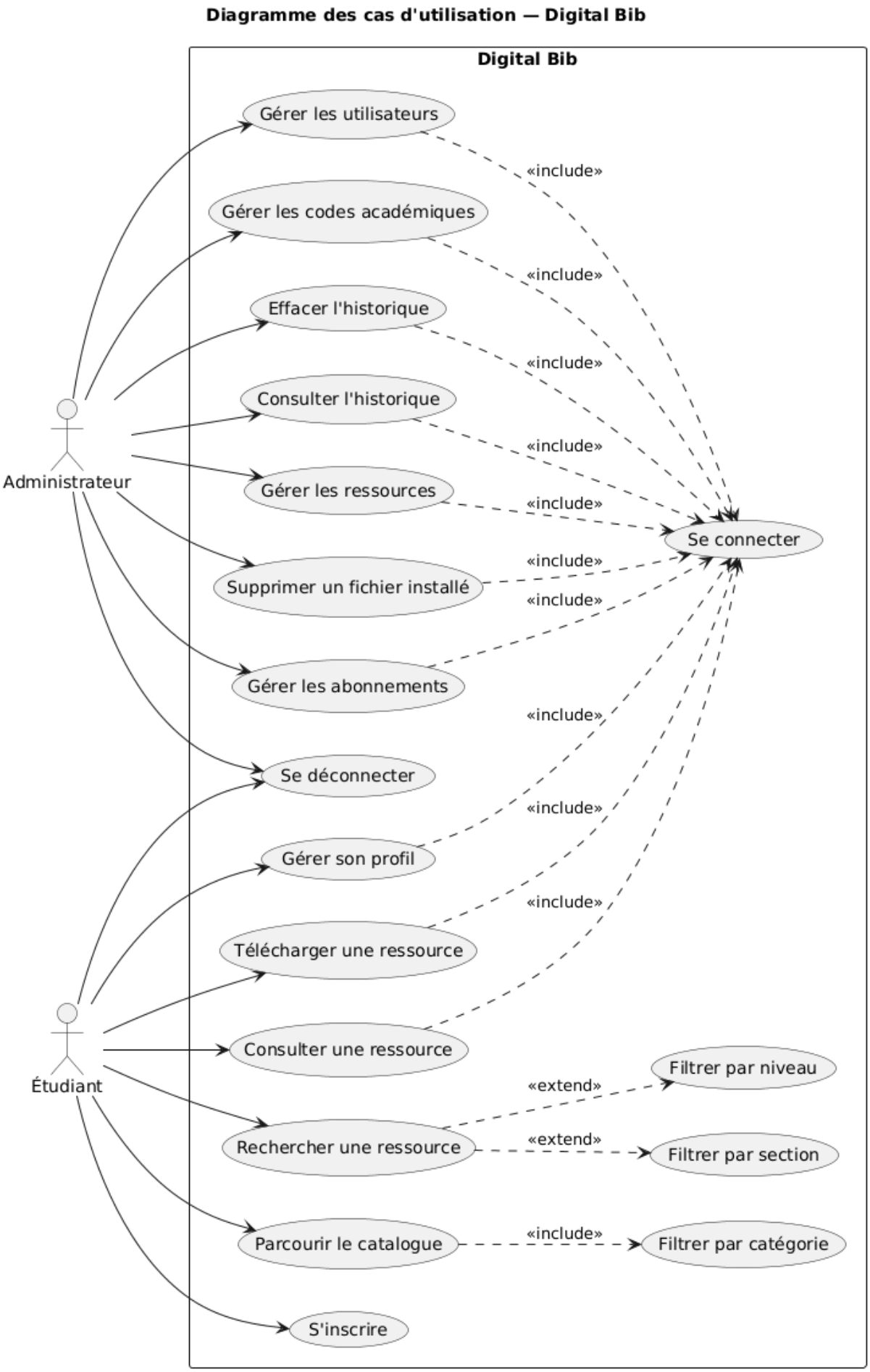


FIGURE 2.1 – Diagramme des cas d'utilisation — Digital Bib

2.2 Détail des cas d'utilisation

2.2.1 UC1 : Rechercher une ressource

- **Préconditions :** L'utilisateur est connecté à la plateforme.
- **Postconditions :** Une liste filtrée de ressources s'affiche.
- **Scénario nominal :**
  1. L'utilisateur accède à la page principale.
  2. Il saisit un mot-clé dans la barre de recherche.
  3. Il sélectionne optionnellement une section et un niveau.
  4. Le système interroge `GET /api/ressources?q=&section=&niveau=`.
  5. Les résultats filtrés s'affichent sous forme de cartes.
- **Scénario alternatif :** Aucun résultat trouvé ⇒ message "Aucune ressource trouvée".

2.2.2 UC2 : Télécharger une ressource

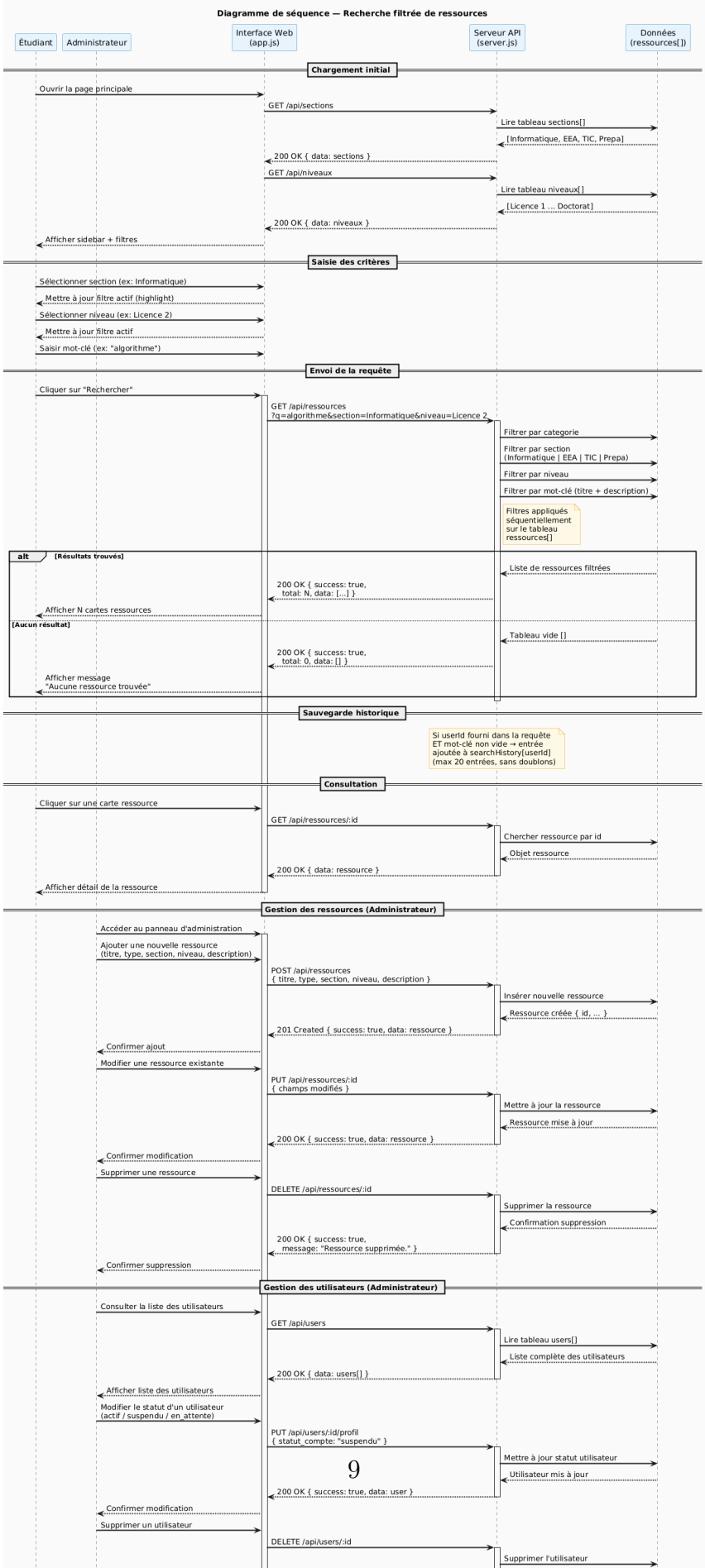
- **Préconditions :** L'utilisateur est connecté et a trouvé la ressource souhaitée.
- **Postconditions :** La ressource est ajoutée à la liste des fichiers installés.
- **Scénario nominal :**
  1. L'utilisateur clique sur *Télécharger* d'une ressource.
  2. Le système envoie `POST /api/users/:id/fichiers`.
  3. La ressource est enregistrée (sans doublon) dans la liste personnelle.
  4. Un message de confirmation s'affiche.
- **Scénario alternatif :** Ressource déjà téléchargée ⇒ doublon ignoré.

2.2.3 UC3 : S'authentifier

- **Préconditions :** L'utilisateur possède un compte valide.
- **Postconditions :** L'utilisateur est redirigé vers la page principale.
- **Scénario nominal :**
  1. L'utilisateur accède à la page de connexion.
  2. Il saisit son adresse e-mail et son mot de passe.
  3. Le système envoie `POST /api/auth/login`.
  4. Un token d'authentification est retourné et stocké côté client.
  5. L'utilisateur est redirigé vers l'accueil.
- **Scénario alternatif :** Identifiants incorrects ⇒ message d'erreur 401.

2.3 Diagrammes de séquence

2.3.1 Recherche filtrée de ressources



2.3.2 Authentification

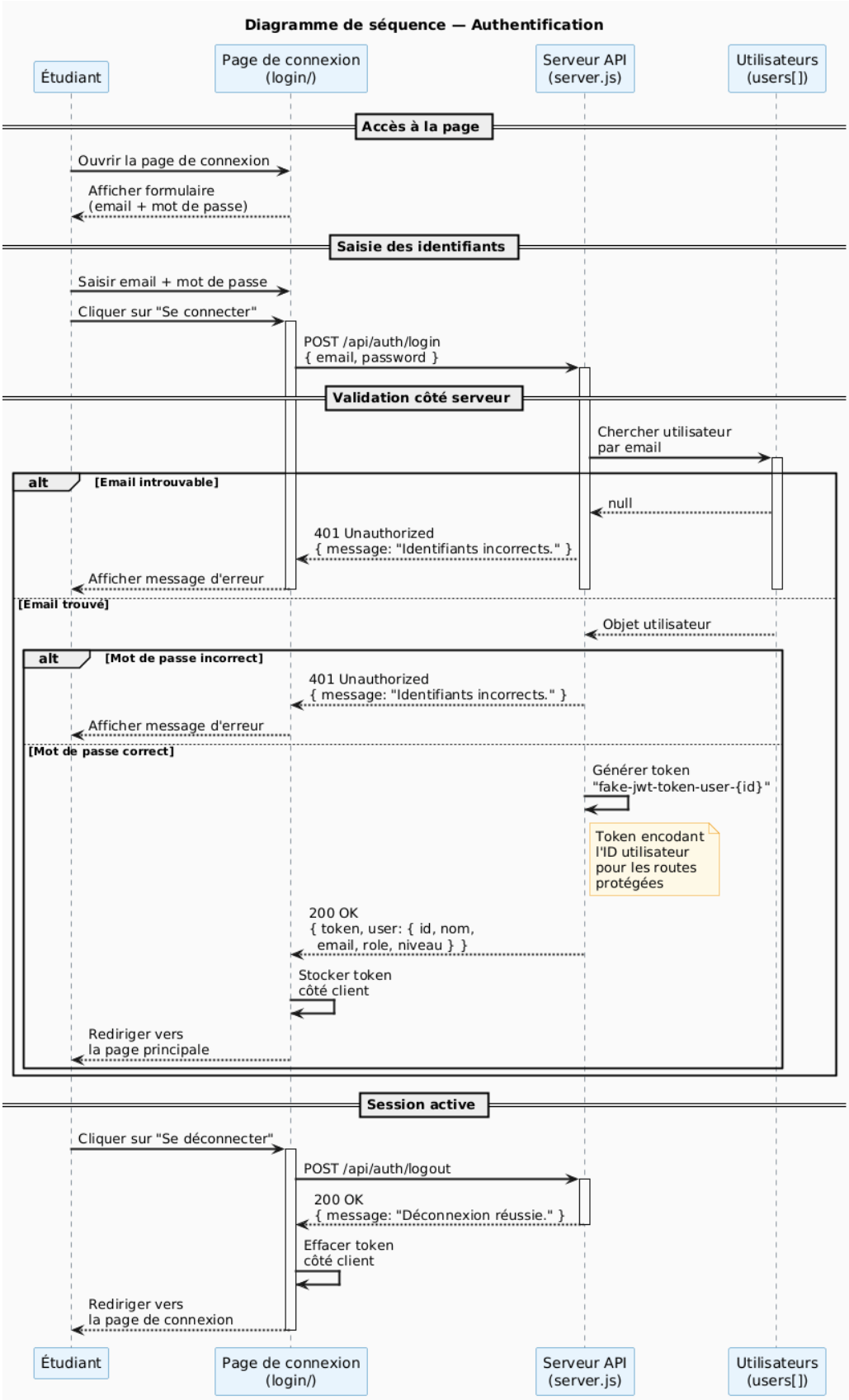


FIGURE 2.3 – Diagramme de séquence — Authentification

2.4 Diagramme de classes

Les principales entités du système :

— **Utilisateur** : id, prenom, nom, email, password, type_utilisateur, statut_compte, date_inscription.
— **Ressource** : id, titre, type, section, niveau, description.
— **CodeAcademique** : code_valide, est_utilise, date_generation.
— **Abonnement** : id, user_id, date_debut, date_fin_validite, montant_paye, statut_paiement.
— **Historique** : userId, query, categorie, date.
— **FichierInstalle** : ressourceId, titre, type, dateTelechargement.

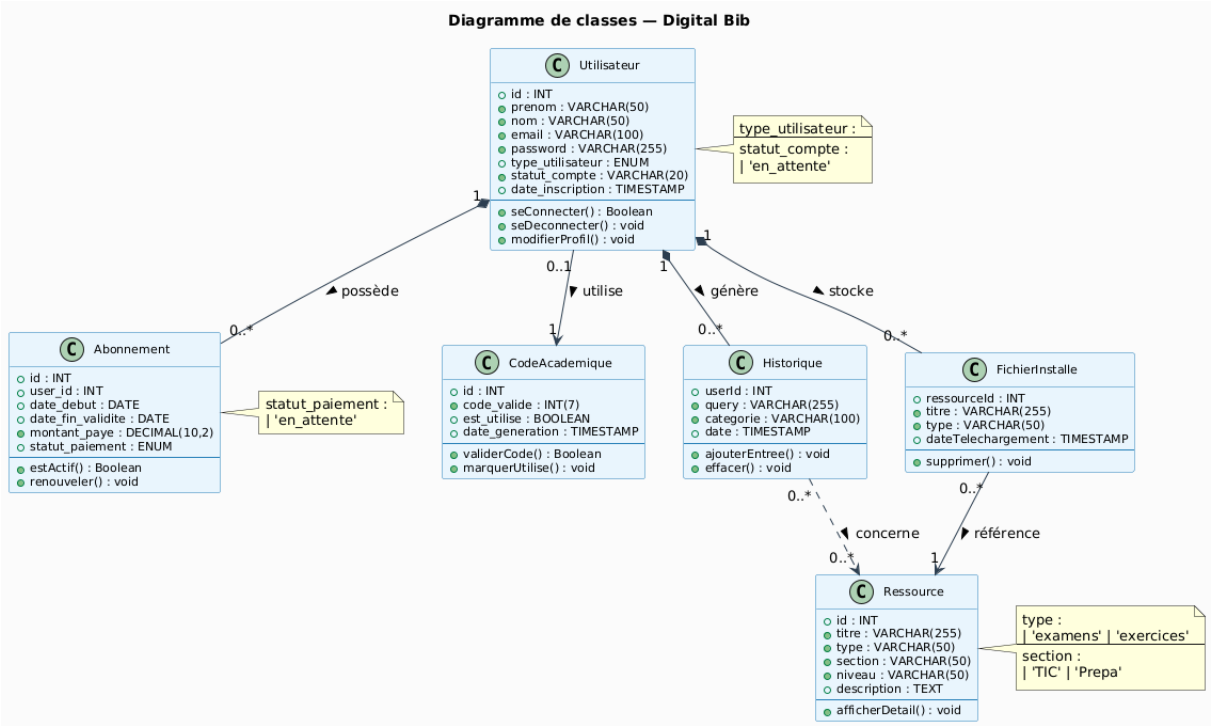


FIGURE 2.4 – Diagramme de classes — Digital Bib

2.5 Architecture technique

L'architecture de **Digital Bib** suit le modèle client-serveur en trois couches :

— **Front-end (Client)** : HTML5 / CSS3 / JavaScript Vanilla. Communique avec le back-end via `fetch` (requêtes HTTP/JSON).
— **Back-end (Serveur)** : Node.js + Express.js sur le port 3000. Expose une API REST, applique la logique métier (filtres, historique, gestion des fichiers) et retourne des réponses JSON.
— **Base de données** : MySQL (`faculte_db`) pour la persistance des utilisateurs, codes académiques et abonnements.

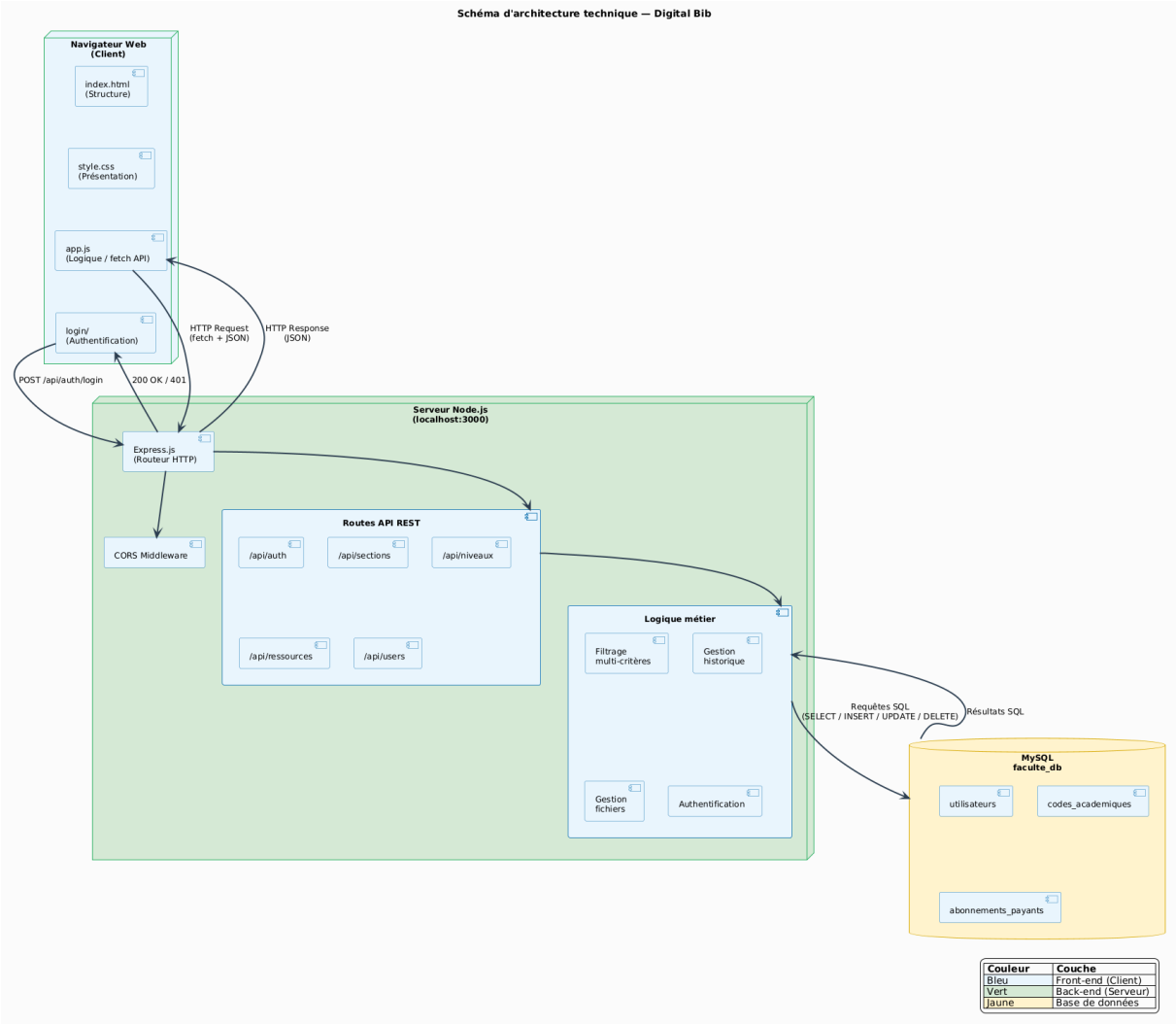


FIGURE 2.5 – Schéma d'architecture technique — Digital Bib

2.6 Maquettes / Wireframes

Choix ergonomiques retenus :

— Barre de recherche centrale et visible pour un accès rapide.
— Filtres (section, niveau, catégorie) dans une barre latérale fixe.
— Affichage des ressources en cartes pour faciliter le survol visuel.
— Indicateur visuel sur chaque carte (téléchargé / non téléchargé).

Chapitre 3

Développement

3.1 Technologies choisies et justifications

- **Front-end : HTML5 / CSS3 / JavaScript (Vanilla)** — Choix léger sans dépendance à un framework lourd, compatibilité maximale avec tous les navigateurs.
- **Back-end : Node.js + Express.js** — Exécution non bloquante idéale pour une API REST à fort trafic. Express simplifie la définition des routes et des middlewares. Le package `cors` autorise les requêtes cross-origin.
- **Base de données : MySQL** — Système relationnel adapté à la structure des données (utilisateurs, abonnements, codes académiques avec contraintes d'intégrité).
- **nodemon (dev)** — Redémarrage automatique du serveur lors des modifications, accélérant le cycle de développement.

3.2 Organisation du code

```
projet-DEV-WEB/
  server.js         # Serveur Express + toutes les routes API REST
  database.js       # Configuration de la connexion MySQL
  database.sql      # Script de creation de la base de donnees
  app.js            # Logique front-end (fetch, filtres, rendu DOM)
  index.html        # Interface principale (SPA)
  style.css         # Styles globaux de l'interface
  login/            # Page et logique de connexion
  API.md            # Documentation complete des endpoints
  package.json      # Dependances et scripts npm
  .gitignore
```

Listing 3.1 – Arborescence du projet Digital Bib

3.3 Implémentation des fonctionnalités clés

3.3.1 Recherche et filtrage des ressources

Logique : Le front-end construit dynamiquement l'URL selon les critères sélectionnés (mot-clé, section, niveau, catégorie) et envoie une requête GET. Le serveur filtre séquentiellement le tableau de ressources et retourne le résultat JSON.

```
app.get("/api/ressources", (req, res) => {
  const { q = "", categorie = "Tous",
          section, niveau, userId } = req.query;
  let results = [...ressources];

  if (categorie !== "Tous") {
    const typeKey = typeMap[categorie];
    if (typeKey) results = results.filter(r => r.type === typeKey);
  }
  if (section) results = results.filter(r => r.section === section);
  if (niveau)  results = results.filter(r => r.niveau  === niveau);
  if (q.trim()) {
    const query = q.toLowerCase();
    results = results.filter(r =>
      r.titre.toLowerCase().includes(query) ||
      r.description.toLowerCase().includes(query)
    );
  }
  // Historique : sauvegarde si userId fourni
  if (userId && q.trim()) {
    if (!searchHistory[userId]) searchHistory[userId] = [];
    const entry = { query: q, categorie,
                    date: new Date().toISOString() };
    const last = searchHistory[userId].at(-1);
    if (!last || last.query !== q) {
      searchHistory[userId].unshift(entry);
      if (searchHistory[userId].length > 20)
        searchHistory[userId].pop();
    }
  }
  res.json({ success: true, total: results.length, data: results });
});
```

Listing 3.2 – Route GET /api/ressources avec filtres multi-critères

Difficulté : Appliquer plusieurs filtres sans erreur de logique.
Solution : Filtres appliqués séquentiellement sur une copie du tableau.

3.3.2 Authentification

Logique : Le client POST les identifiants, le serveur les vérifie et retourne un token ainsi que les données utilisateur.

```
app.post("/api/auth/login", (req, res) => {
  const { email, password } = req.body;
  if (!email || !password)
    return res.status(400).json({ success: false,
      message: "Email et mot de passe requis." });

  const user = users.find(u => u.email === email);
  if (!user || password !== "1234")
    return res.status(401).json({ success: false,
      message: "Identifiants incorrects." });

  res.json({ success: true, message: "Connexion reussie.",
    data: { token: `fake-jwt-token-user-${user.id}`, user }
  });
```

```
});
```

Listing 3.3 – Route POST /api/auth/login

Difficulté : Sécuriser les routes selon le rôle utilisateur.
Solution : Le token encode l'ID utilisateur ; les routes protégées le vérifient avant d'exécuter la logique métier.

3.3.3 Gestion des fichiers téléchargés

Logique : Avant d'ajouter une ressource à la liste personnelle, le système vérifie l'absence de doublon par recherche dans le tableau existant.

```
app.post("/api/users/:id/fichiers", (req, res) => {
  const { ressourceId } = req.body;
  const ressource = ressources.find(
    r => r.id === parseInt(ressourceId));
  if (!ressource) return res.status(404).json(
    { success: false, message: "Ressource introuvable." });

  const userId = req.params.id;
  if (!fichiersTelecharges[userId])
    fichiersTelecharges[userId] = [];

  const exists = fichiersTelecharges[userId]
    .find(f => f.ressourceId === ressource.id);
  if (!exists) {
    fichiersTelecharges[userId].push({
      ressourceId: ressource.id,
      titre: ressource.titre,
      type: ressource.type,
      dateTelechargement: new Date().toISOString()
    });
  }
  res.json({ success: true,
    message: `"${ressource.titre}" ajoute aux fichiers.` });
});
```

Listing 3.4 – Route POST /api/users/:id/fichiers

Difficulté : Éviter les doublons dans la liste personnelle.
Solution : Vérification par `Array.find()` avant chaque insertion.

3.4 Gestion des données

3.4.1 Schéma de la base de données

La base `faculte_db` comprend trois tables :

- **codes_academiques** : codes à 7 chiffres générés par l'administration pour l'inscription des étudiants internes.
- **utilisateurs** : données personnelles, type (*interne/étranger*), statut et référence au code d'inscription.
- **abonnements_payants** : gestion des accès étrangers avec dates de validité et statut de paiement.

```sql
CREATE TABLE utilisateurs (
    id                        INT AUTO_INCREMENT PRIMARY KEY,
    prenom                    VARCHAR(50) NOT NULL,
    nom                       VARCHAR(50) NOT NULL,
    email                     VARCHAR(100) NOT NULL UNIQUE,
    password                  VARCHAR(255) NOT NULL,
    type_utilisateur          ENUM('interne','etranger') NOT NULL,
    code_inscription_utilise  INT(7) NULL,
    statut_compte             VARCHAR(20) DEFAULT 'actif',
    date_inscription          TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_code
        FOREIGN KEY (code_inscription_utilise)
        REFERENCES codes_academiques(code_valide)
        ON DELETE SET NULL
);
```

Listing 3.5 – Extrait du script database.sql

```sql
SELECT u.nom, u.email, a.date_fin_validite
FROM   utilisateurs u
JOIN   abonnements_payants a ON u.id = a.user_id
WHERE  a.date_fin_validite >= CURDATE()
  AND  a.statut_paiement   = 'succes'
ORDER  BY a.date_fin_validite ASC;
```

Listing 3.6 – Requête : abonnements actifs

Chapitre 4

Tests et validation

4.1 Environnement et outils de test

— **Navigateurs testés :** Google Chrome, Mozilla Firefox, Microsoft Edge.
— **Outil de test API :** Postman et commandes `curl`.
— **Validateurs :** W3C Validator (HTML/CSS).
— **Environnement :** Node.js v18, npm, Windows 11.

4.2 Scénarios de test

Table 4.1 – Résultats des tests fonctionnels

Fonctionnalité	**Statut**	**Observations**
Connexion (login)	OK	Token retourné correctement
Déconnexion (logout)	OK	Session côté client effacée
Lister les sections	OK	4 sections retournées
Lister les niveaux	OK	Licence 1 à Doctorat
Recherche par mot-clé	OK	Filtre titre et description
Filtrage par section	OK	Résultats cohérents
Filtrage par niveau	OK	Résultats cohérents
Téléchargement d'une ressource	OK	Ajout sans doublon vérifié
Suppression d'un fichier installé	OK	Suppression correcte par ID
Historique de recherche	OK	Limité à 20 entrées
Effacement de l'historique	OK	Tableau réinitialisé
Mise à jour du profil	OK	Champs modifiés en mémoire
Route inconnue (404)	OK	Message JSON d'erreur retourné

Chapitre 5

Conclusion et perspectives

5.1 Bilan du projet

Le projet **Digital Bib** a atteint ses objectifs principaux :

— Une API REST complète et documentée (15 endpoints) couvrant toutes les fonctionnalités prévues.
— Une interface front-end fonctionnelle permettant la recherche, le filtrage multi-critères et la gestion des ressources.
— Un système d'authentification, de gestion de profil et d'historique de recherche opérationnel.

5.2 Difficultés et apprentissages

— **Coordination en équipe :** gestion de plusieurs branches Git simultanées nécessitant une organisation rigoureuse des merges.
— **Données en mémoire :** absence de persistance réelle — le serveur réinitialise l'état à chaque redémarrage.
— **Apprentissages :** maîtrise de Node.js/Express, du protocole REST, de la conception de bases de données relationnelles (MySQL), et du travail collaboratif via Git.

5.3 Évolutions possibles

— Connexion effective à la base MySQL pour une persistance totale.
— Authentification JWT sécurisée (signature, expiration).
— Interface administrateur pour gérer le catalogue de ressources.
— Système de notation et de commentaires sur les ressources.
— Déploiement cloud (Railway, Render ou VPS dédié).
— Application mobile consommant la même API REST.

Chapitre 6

Annexes

6.1 Lien vers le dépôt Git

`https://github.com/hmaida11/projet-DEV-WEB`

6.2 Tableau récapitulatif des endpoints API

Table 6.1 – Endpoints REST — Digital Bib

Méthode	**Route**	**Description**
GET	/api/sections	Liste des sections
GET	/api/niveaux	Liste des niveaux
GET	/api/categories	Liste des catégories
GET	/api/ressources	Ressources (avec filtres)
GET	/api/ressources/ :id	Détail d'une ressource
GET	/api/ressources/type/ :type	Par type (cours, td, ...)
POST	/api/auth/login	Connexion
POST	/api/auth/logout	Déconnexion
GET	/api/users/ :id/profil	Récupérer le profil
PUT	/api/users/ :id/profil	Modifier le profil
GET	/api/users/ :id/historique	Historique de recherche
DELETE	/api/users/ :id/historique	Effacer l'historique
GET	/api/users/ :id/fichiers	Fichiers téléchargés
POST	/api/users/ :id/fichiers	Ajouter un fichier
DELETE	/api/users/ :id/fichiers/ :rid	Supprimer un fichier

6.3 Fichiers de configuration

```
{
  "name": "faculte-backend",
  "version": "1.0.0",
  "description": "Backend API REST pour Digital Bib",
  "main": "server.js",
  "scripts": {
    "start": "node server.js",
```

```
    "dev":   "nodemon server.js"
  },
  "dependencies": {
    "cors":    "^2.8.5",
    "express": "^4.18.2"
  },
  "devDependencies": {
    "nodemon": "^3.0.1"
  }
}
```

Listing 6.1 – package.json du projet